ICFAI Foundation for Higher Education
(Deemed-to-be University under Section 3 of the UGC Act, 1956)

A

Computer Vision Course Assignment Project Report

On

Comparative Analysis of Two-Stage (R-CNN) vs. One-Stage (YOLO) Detectors for Speed and Accuracy

Submitted by:

{{NAME_1}} ({{REGD_NO_1}})

{{NAME_2}} ({{REGD_NO_2}})

{{NAME_3}} ({{REGD_NO_3}})

{{NAME_4}} ({{REGD_NO_4}})

{{PROGRAM}}

Faculty Name: {{FACULTY_NAME}}

IcfaiTech
Faculty of Science & Technology (FST)

Faculty of Science & Technology

IFHE University, Hyderabad

Nov 2026

ABSTRACT

Object detectors fall into two broad families. Two-stage detectors such as Faster R-CNN first propose candidate regions and then classify and refine each region, while one-stage detectors such as YOLO predict classes and boxes for the whole image in a single forward pass. This project measures the practical speed-accuracy trade-off between the two families under identical conditions. Five pretrained COCO models were considered: Faster R-CNN with ResNet-50-FPN (v2) and MobileNetV3-Large-FPN backbones (torchvision), and YOLOv8 n/s/m (Ultralytics). We benchmarked 4 of them (FasterRCNN-R50-FPN-v2, FasterRCNN-MobileNetV3-FPN, YOLOv8n, YOLOv8s) on the first 100 images of COCO val2017 on CPU (x86_64, 4 threads), measuring per-image end-to-end latency at batch size 1 after a warm-up, frames per second, parameter count and weight size, and accuracy with the official COCO evaluation (mAP@[.5:.95], mAP50, mAP75 and mAP for small, medium and large objects). YOLOv8s ran 33.2x faster than Faster R-CNN R50-FPN v2 (7.52 vs 0.23 FPS) with an mAP of 0.538 against 0.560. The most accurate model in our run was FasterRCNN-R50-FPN-v2 (mAP 0.560) and the fastest was YOLOv8n (18.7 FPS). Small objects were the hardest category for every model; the best small-object mAP was 0.401 (FasterRCNN-R50-FPN-v2). We also built a Streamlit application that runs both detectors side by side on images, video and a webcam and reports their inference time live. The results show that a modern one-stage detector now offers a better speed-accuracy balance for real-time use, while the two-stage design remains a strong, well-understood baseline whose region-based second stage is costly on resource-limited hardware.

Keywords: Object detection, Faster R-CNN, YOLOv8, Region Proposal Network, mean Average Precision (mAP), COCO dataset, Inference latency, Speed-accuracy trade-off

TABLE OF CONTENTS

S. No.	Section Title	Page No.
1	**Introduction**	**4**
1.1	Background of the Project	4
1.2	Problem Definition	4
1.3	Objectives	4
2	**Case Design / Algorithm Implementation**	**6**
2.1	Dataset	6
2.2	Tools, Languages and Frameworks	6
2.3	Two-Stage Detector: Faster R-CNN	6
2.4	One-Stage Detector: YOLOv8	7
2.5	Evaluation Metrics	8
2.6	Benchmarking Methodology	8
2.7	Key Code Snippets	9
2.8	Modules and Live Demo Application	10
3	**Results and Discussion**	**12**
3.1	Quantitative Results	12
3.2	Speed-Accuracy Trade-off	14
3.3	Small-Object Performance	15
3.4	Qualitative Results	16
3.5	Comparison with Published Results	18
3.6	Challenges Faced and Solutions	18
4	**Conclusion**	**19**
4.1	Summary of What Was Achieved	19
4.2	Limitations	19
4.3	Possible Improvements	19
	References	**21**
	Appendix	**22**
A.	Source Code	22
B.	How to Run	22
C.	Project Structure	22
D.	Experiment Environment	22

1. Introduction

1.1 Background of the Project

Object detection asks a model to find every object of interest in an image, draw a bounding box around it and name its class. It underpins autonomous driving, video surveillance, robotics, retail analytics and medical imaging. Since 2014 deep convolutional networks have dominated the task, and they have evolved along two lines. The R-CNN family [1], [5] (R-CNN, Fast R-CNN, Faster R-CNN) is **two-stage**: a region proposal step selects a few hundred to a few thousand candidate boxes and a second network classifies and refines each one. The YOLO family [2] (You Only Look Once) is **one-stage**: it treats detection as a single regression problem and predicts all boxes and classes in one pass over the image. The common belief is that two-stage detectors are more accurate, especially on small objects, while one-stage detectors are faster. Both families have changed a lot since their first papers, so the belief needs to be checked with current models.

1.2 Problem Definition

Given a fixed set of pretrained detectors, a fixed set of test images and fixed hardware, we want to quantify how much accuracy is gained or lost and how much speed is gained or lost when moving from a two-stage to a one-stage detector. The comparison must be fair: the same images, batch size 1 (as in a live video stream), the same evaluation code, the same score threshold, and timing that excludes one-time start-up costs. The question we answer is: **for a real-time application on our hardware, which detector family gives the better trade-off, and where does each one fail?**

1.3 Objectives

- Benchmark two-stage detectors (Faster R-CNN ResNet-50-FPN v2 and MobileNetV3-Large-FPN) and one-stage detectors (YOLOv8n, YOLOv8s, YOLOv8m) with pretrained COCO weights and no retraining.
- Measure speed: mean and standard deviation of per-image latency, FPS, parameter count and weight size.
- Measure accuracy with the official COCO protocol: mAP@[.5:.95], mAP50, mAP75 and mAP by object size.
- Analyse the speed-accuracy trade-off and small-object behaviour, supported by charts and visual examples.

- Build a live demo that runs both detectors side by side on images, video and a webcam.

2. Case Design / Algorithm Implementation

2.1 Dataset

We use **COCO 2017 validation** [3] (5,000 images, 80 object classes). Because the experiments run on a CPU, we evaluate on the first **100 images** (sorted by image id), which contain **709 ground-truth boxes**. Only the image files we need are downloaded, and only instances_val2017.json is extracted from the annotation archive (with HTTP range requests, so the 241 MB zip is never stored on disk). All models were trained on COCO train2017, so val2017 is unseen data for every model.

2.2 Tools, Languages and Frameworks

Table 1: Software stack (versions recorded by benchmark.py)

Component	Used for	Version
Python	All code	3.13.5
PyTorch	Tensor computation, inference	2.15.0+cpu
torchvision	Faster R-CNN models and weights	0.29.2+cpu
Ultralytics	YOLOv8 models and weights	8.4.207
pycocotools	Official COCO mAP evaluation	2.0.11
OpenCV, NumPy	Image I/O, drawing, arrays	5.0.0, 2.5.4
Matplotlib	Charts and diagrams	-
Streamlit	Live demo web application	-
python-docx, python-pptx	Generating this report and the slides	-

2.3 Two-Stage Detector: Faster R-CNN

Faster R-CNN [1] has four parts (Figure 1, top row):

- **Backbone:** a CNN (ResNet-50 or MobileNetV3) pretrained on ImageNet that turns the image into feature maps.
- **Feature Pyramid Network (FPN) [4]:** combines deep, semantically strong but coarse features with shallow, high-resolution features through a top-down path with lateral connections. Each pyramid level handles a range of object sizes, which helps with small objects.
- **Region Proposal Network (RPN), stage 1:** a small convolutional network slides over every pyramid level. At each position it places several **anchors** (reference boxes of different sizes and aspect ratios) and predicts an objectness score and four box offsets for each anchor. After NMS the top proposals (about 1,000 at test time) are kept.
- **RoI head, stage 2:** RoIAlign (the improved form of RoI pooling [5]) cuts a fixed 7x7 feature grid out of the feature map for every proposal using bilinear interpolation. Fully

connected layers then predict the class (81 outputs including background) and a class-specific box refinement. Per-class NMS gives the final detections.

We use torchvision's **v2** weights for the ResNet-50 model, which were trained with a modern, longer recipe [8] and reach a higher mAP than the original 2015 configuration.

The second stage is run once **per proposal**, so its cost grows with the number of proposals. torchvision also resizes images so the short side is 800 pixels, which helps accuracy but costs time.

2.4 One-Stage Detector: YOLOv8

YOLO [2] removes the proposal stage. YOLOv8 [6] works as follows (Figure 1, bottom row):

- **Backbone:** CSPDarknet with C2f blocks (cross-stage partial connections) to extract features cheaply.
- **Neck:** a PAN-FPN that fuses features top-down and bottom-up, producing three scales (strides 8, 16, 32).
- **Decoupled, anchor-free head:** every grid cell on every scale directly predicts class probabilities in one branch and box geometry in another branch, as distances from the cell centre to the four box sides (Distribution Focal Loss). There are no hand-designed anchors.
- **Post-processing:** a confidence threshold and NMS. The whole network runs once per image at 640x640.

Because everything is a single dense convolutional pass with no per-region computation, YOLO is well suited to real-time video. The n/s/m variants share the design and differ only in depth and width.

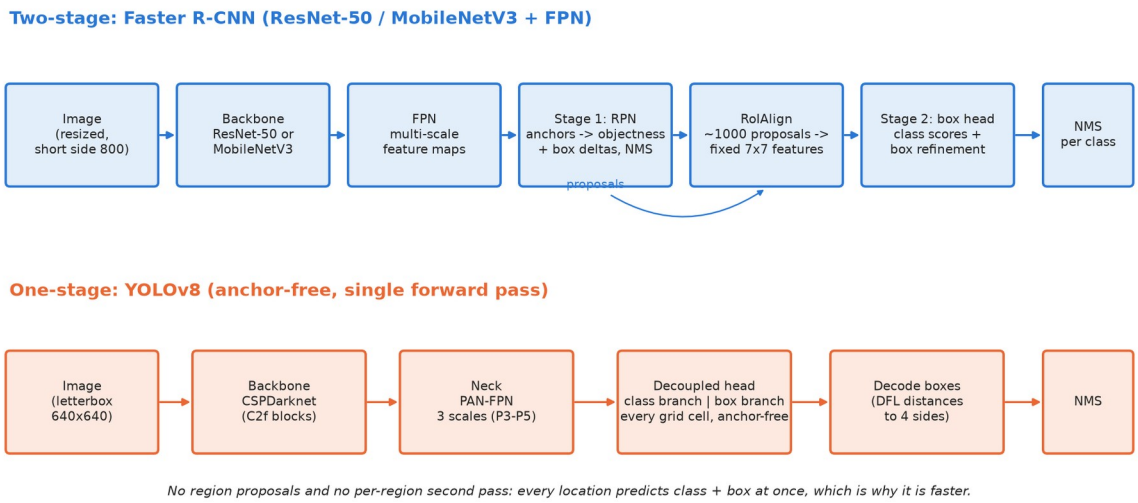


Figure 1: Pipelines of the two detector families compared in this project

2.5 Evaluation Metrics

- **IoU (Intersection over Union):** area of overlap between predicted and ground-truth box divided by the area of their union. A detection is a true positive if IoU is at least a threshold and the class is right.
- **Precision** = TP / (TP + FP), the share of detections that are correct; **Recall** = TP / (TP + FN), the share of objects that were found.
- **AP (Average Precision):** the area under the precision-recall curve of one class, which is obtained by sweeping the score threshold. COCO uses 101-point interpolation [9].
- **mAP:** AP averaged over the 80 classes. **mAP@[.5:.95]** (the main COCO metric) also averages over ten IoU thresholds from 0.50 to 0.95, which rewards precise localisation. **mAP50** and **mAP75** use a single IoU threshold. **mAP small / medium / large** restrict evaluation to objects with area below 32x32 pixels, between 32x32 and 96x96, and above 96x96.
- **Latency** is the wall-clock time for one image (preprocessing + network + NMS); **FPS = 1000 / mean latency in ms**.

2.6 Benchmarking Methodology

- Batch size 1 and the default input size of each model (torchvision: short side 800 (max 1333); YOLO: 640 letterbox).
- 10 warm-up images per model, not timed (first calls include memory allocation and lazy initialisation).

- torch.cuda.synchronize() before and after each timed call on GPU, because CUDA kernels run asynchronously.
- Image decoding from disk is excluded from the timing; end-to-end per image preprocessing + forward + post-processing (NMS); image decoding excluded.
- Score threshold 0.001 and at most 100 detections per image for evaluation, so that the full precision-recall curve can be computed.
- YOLO class indices 0-79 are mapped to the sorted COCO category ids (1-90 with gaps); torchvision already outputs COCO ids. benchmark.py asserts that this mapping matches the annotation file.
- **Hardware note:** No CUDA GPU found: ran on CPU with N=100 images and skipped YOLOv8m (largest YOLO model).

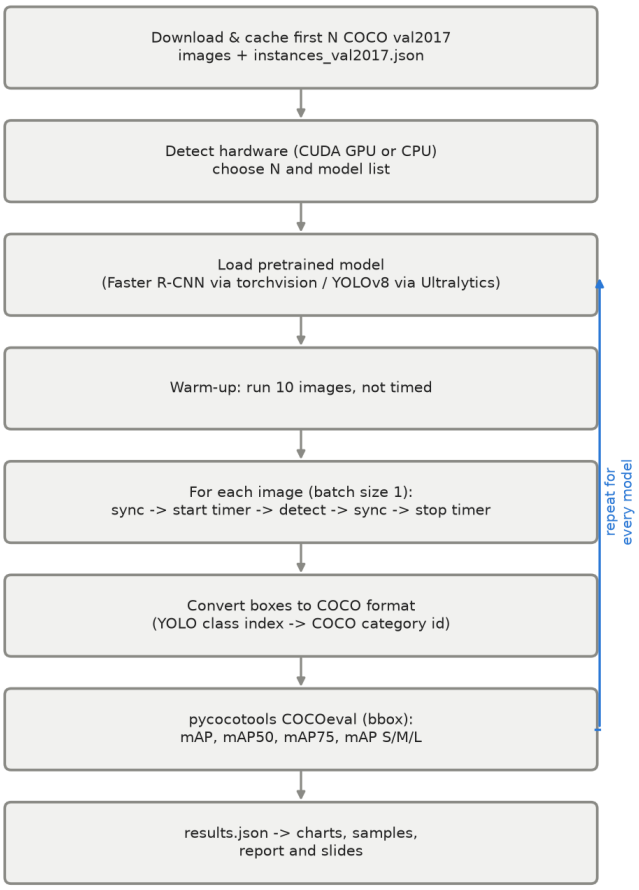


Figure 2: Flowchart of the benchmarking procedure (benchmark.py)

2.7 Key Code Snippets

A single function hides the differences between the two libraries, so both families go through exactly the same timing and evaluation code (common.py):

```
@torch.no_grad()
def detect(info, image_bgr):

    if info["kind"] == "torchvision":
        rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
        tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255).to(info["device"])
        out = info["model"]([tensor])[0]          # model resizes internally (short
side 800)
        boxes = out["boxes"].cpu().numpy()
        scores = out["scores"].cpu().numpy()
        coco_ids = out["labels"].cpu().numpy().astype(int)
        names = [info["names"][c] for c in coco_ids]
    else:
        res = info["model"].predict(image_bgr, conf=info["score_thresh"], iou=0.7,
imgsz=640,
                                    max_det=info["max_det"], device=info["device"],
verbose=False)[0]
        boxes = res.boxes.xyxy.cpu().numpy()
        scores = res.boxes.conf.cpu().numpy()
        cls = res.boxes.cls.cpu().numpy().astype(int)
        coco_ids = np.array([COCO_IDS[c] for c in cls], dtype=int)   # YOLO index ->
COCO id
        names = [info["names"][c] for c in cls]
    return boxes, scores, coco_ids, names
```

Timing with synchronisation (common.py) and conversion to COCO format plus evaluation (benchmark.py):

```
def timed_detect(info, image_bgr):
    sync(info["device"])
    t0 = time.perf_counter()
    result = detect(info, image_bgr)
    sync(info["device"])
    return result, (time.perf_counter() - t0) * 1000.0
```

```
# COCO wants [x, y, width, height]; one dict per detected box
dets.append({"image_id": img_id, "category_id": int(c),
             "bbox": [x1, y1, x2 - x1, y2 - y1], "score": float(s)})

    with contextlib.redirect_stdout(io.StringIO()):
        coco_dt = coco.loadRes(dets)
        ev = COCOeval(coco, coco_dt, "bbox")
        ev.params.imgIds = img_ids                # only score the images we actually
ran
        ev.evaluate(); ev.accumulate(); ev.summarize()
    out = {k: float(ev.stats[i]) for i, k in enumerate(keys)}
    out["AR100"] = float(ev.stats[8])
    return out
```

2.8 Modules and Live Demo Application

Table 2: Project modules

File	Purpose
common.py	Model registry, loading, unified detect(), timing, drawing
benchmark.py	Data download, timing loop, COCO evaluation, charts, sample images, results.json
figures.py	Architecture diagram and methodology flowchart
demo.py	Streamlit side-by-side demo (image, upload, webcam, video) and CLI fallback
build_report.py / build_slides.py	Generate this report and the presentation from results.json

The demo (uv run demo) lets the user choose a two-stage and a one-stage model, an input source (uploaded image, webcam snapshot or live webcam) and a score threshold. Both models run on the same frame and the app shows the annotated outputs next to each other with the inference time, FPS and number of boxes of each model. Models are cached and warmed up once when first selected so the demo responds quickly.

3. Results and Discussion

3.1 Quantitative Results

All results below were measured on **CPU (x86_64, 4 threads)** with **100 COCO val2017 images** (run date 2026-11-08).

Table 3: Speed and accuracy of all models (mAP = COCO mAP@[.5:.95])

Model	Family	Params (M)	Weights (MB)	Latency (ms)	FPS	mAP	mAP50	mAP75
FasterRCNN-R50-FPN-v2	two-stage	43.7	175.2	4411.4 ± 650.6	0.23	0.560	0.770	0.613
FasterRCNN-MobileNetV3-FPN	two-stage	19.4	77.8	735.5 ± 122.7	1.36	0.391	0.594	0.405
YOLOv8n	one-stage	3.2	6.5	53.5 ± 9.1	18.69	0.453	0.621	0.487
YOLOv8s	one-stage	11.2	22.6	133.0 ± 21.8	7.52	0.538	0.693	0.611

Table 4: Accuracy by object size and average recall

Model	mAP small	mAP medium	mAP large	AR@100
FasterRCNN-R50-FPN-v2	0.401	0.601	0.741	0.669
FasterRCNN-MobileNetV3-FPN	0.205	0.450	0.618	0.528
YOLOv8n	0.290	0.470	0.699	0.575
YOLOv8s	0.366	0.568	0.744	0.667

Note on weight size: the Ultralytics .pt files store weights in 16-bit floating point while the torchvision files are 32-bit, so file size is not proportional to parameter count across libraries.

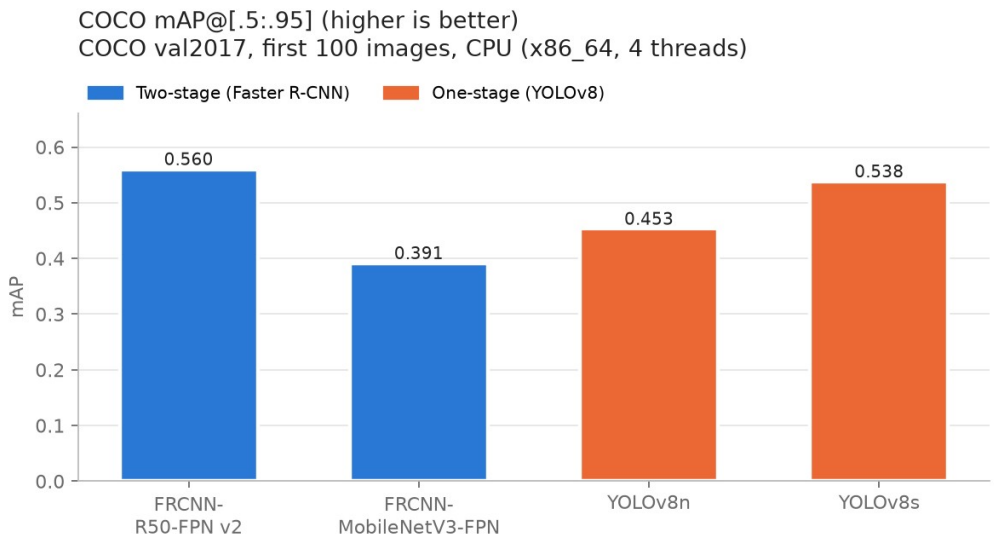


Figure 3: COCO mAP@[.5:.95] per model

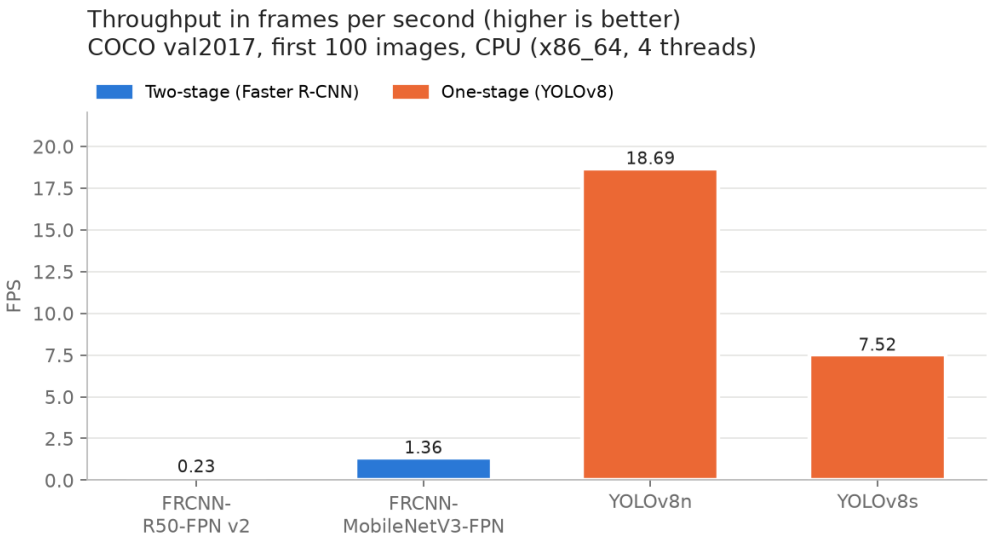


Figure 4: Throughput (FPS) per model

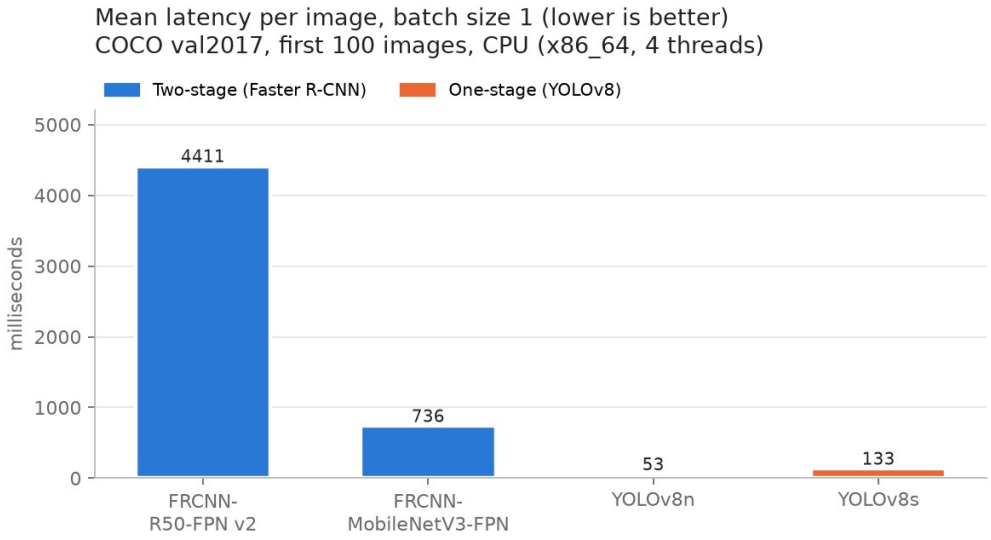


Figure 5: Mean per-image latency per model

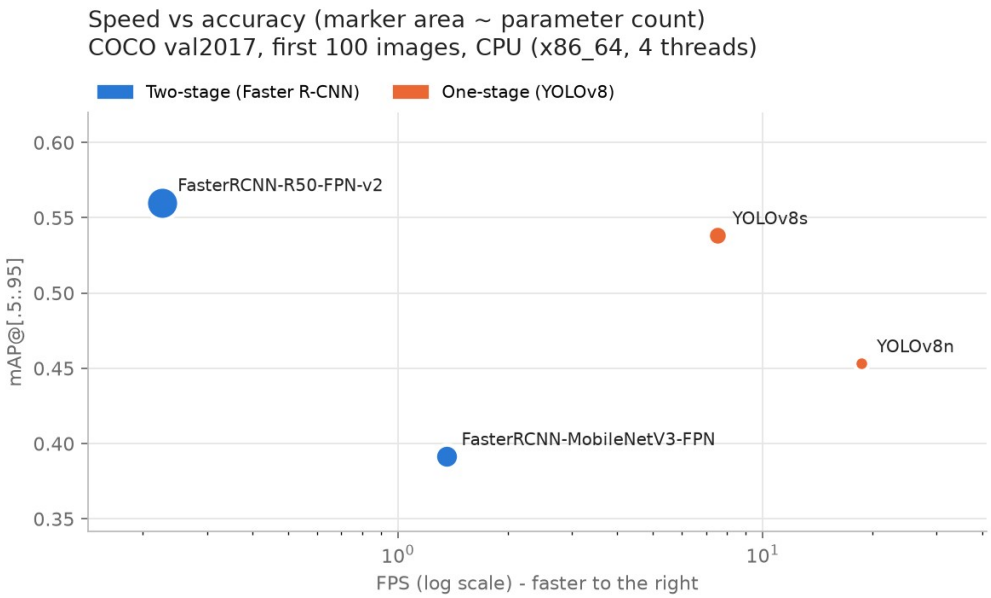


Figure 6: Speed vs accuracy trade-off

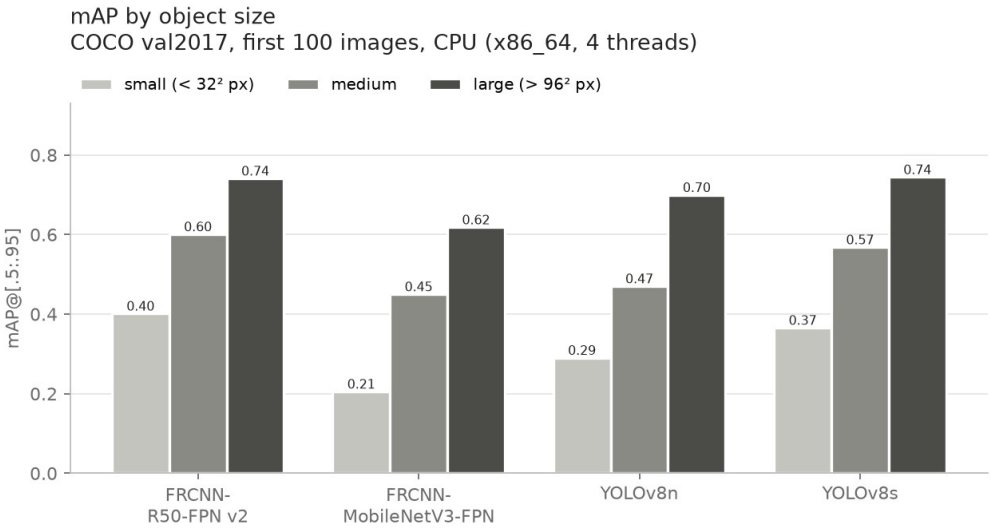


Figure 7: mAP by object size

3.2 Speed-Accuracy Trade-off

From fastest to slowest the models rank: YOLOv8n, YOLOv8s, FasterRCNN-MobileNetV3-FPN, FasterRCNN-R50-FPN-v2. The fastest model, YOLOv8n, runs at 18.7 FPS, which is 82x the throughput of the slowest, FasterRCNN-R50-FPN-v2 (0.23 FPS).

Comparing the main pair, YOLOv8s is 33.2x faster than Faster R-CNN R50-FPN v2 and its mAP is 2.2 points lower (0.538 vs 0.560). It also uses 3.9x fewer parameters.

Even the lighter two-stage model, FasterRCNN-MobileNetV3-FPN, reaches only 1.36 FPS (736 ms per image) with mAP 0.391, while YOLOv8s reaches 7.52 FPS with mAP 0.538.

The Pareto-optimal models (no other model is both faster and more accurate) are: FasterRCNN-R50-FPN-v2, YOLOv8n, YOLOv8s. The front contains both families, so the right choice depends on whether speed or accuracy matters more for the application.

Why is the one-stage detector faster? Faster R-CNN runs its region head separately for each of about 1,000 proposals and processes larger (800-pixel) inputs, whereas YOLOv8 makes all predictions with one dense pass over a 640-pixel input. On a CPU, which has little parallelism compared with a GPU, the extra per-region work and larger input are especially expensive.

3.3 Small-Object Performance

Every model scores far lower on small objects than on large ones (for example FasterRCNN-R50-FPN-v2: 0.401 small vs 0.741 large). The ranking on small objects is FasterRCNN-R50-FPN-v2 (0.401), YOLOv8s (0.366), YOLOv8n (0.290), FasterRCNN-MobileNetV3-FPN (0.205). Here the classical expectation holds: Faster R-CNN R50-FPN v2 beats YOLOv8s on small objects (0.401 vs 0.366). Its larger 800-pixel input keeps more pixels per small object, the FPN gives it a high-resolution P2 level (stride 4, versus stride 8 as the finest YOLOv8 level), and the second stage re-examines each candidate region with RoIAlign.

The MobileNetV3 version of Faster R-CNN shows how much the backbone matters: its small-object mAP is only 0.205. It uses the same 800-pixel input, but its FPN is built only on the last two MobileNetV3 stages (stride 16 and 32 plus one pooled level), so it has no high-resolution feature level for small objects, and its backbone is much weaker. Being two-stage by itself does not guarantee good small-object accuracy.

Caution: with only 100 images the small-object category contains few instances of many classes, so these per-size numbers have a large variance and differences of a few points are not significant.

3.4 Qualitative Results

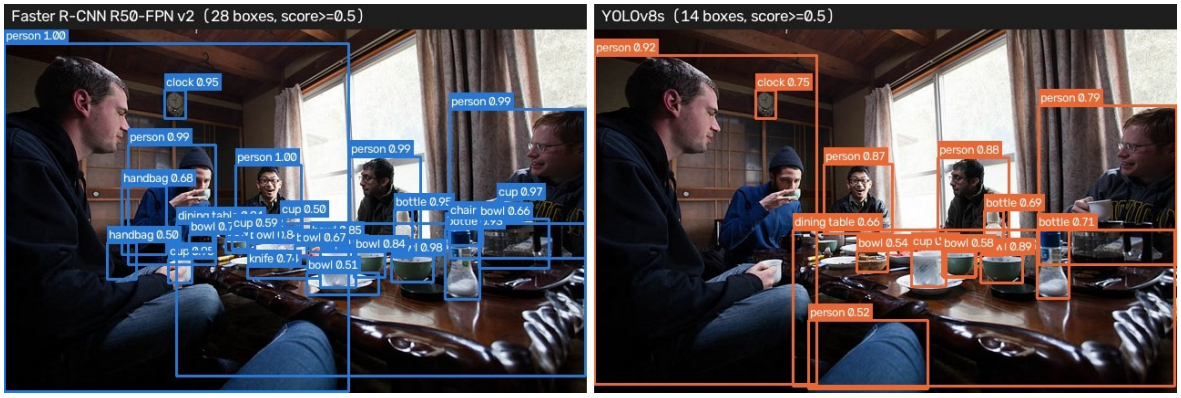


Figure 8: Crowded scene (COCO image 9590, 29 ground-truth objects, 17 small). Left: Faster R-CNN R50-FPN v2, 28 boxes; right: YOLOv8s, 14 boxes (score >= 0.5)

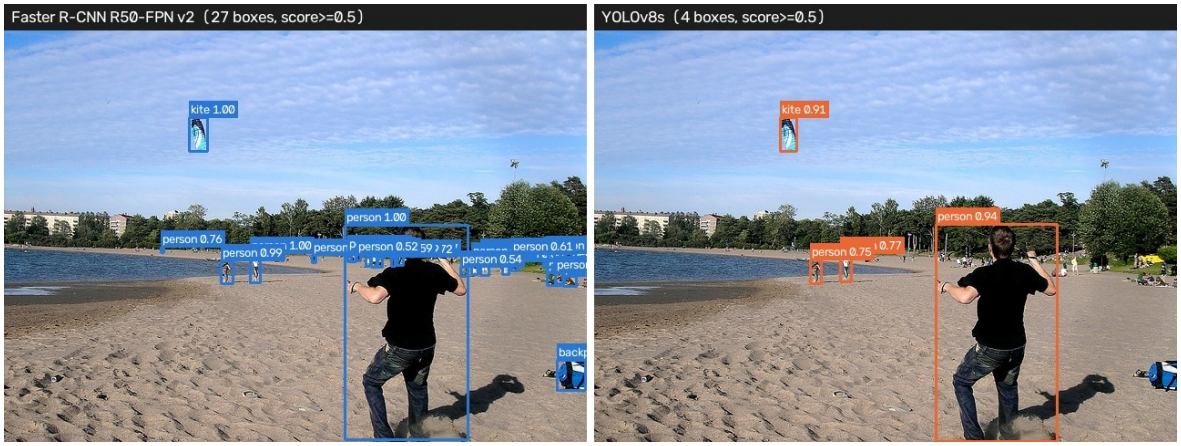


Figure 9: Small objects (COCO image 7511, 16 ground-truth objects, 15 small). Left: Faster R-CNN R50-FPN v2, 27 boxes; right: YOLOv8s, 4 boxes (score >= 0.5)



Figure 10: Typical scene (COCO image 7088, 4 ground-truth objects, 1 small). Left: Faster R-CNN R50-FPN v2, 5 boxes; right: YOLOv8s, 3 boxes (score >= 0.5)



Figure 11: Typical scene (COCO image 8211, 6 ground-truth objects, 2 small). Left: Faster R-CNN R50-FPN v2, 12 boxes; right: YOLOv8s, 4 boxes (score >= 0.5)

In the crowded scene (29 annotated objects), Faster R-CNN produced 28 boxes above 0.5 confidence and YOLOv8s produced 14. Both detectors find the large, clearly visible objects reliably; the differences appear on small, overlapping and partially occluded objects, where boxes are missed or merged by NMS.

The small-object image (15 of 16 objects are small) shows the largest gap: 27 boxes from Faster R-CNN against 4 from YOLOv8s. Faster R-CNN recovers more of the small, distant objects, which agrees with the small-object mAP in Table 4. Two cautions apply to these pictures: the two models calibrate their confidence scores differently (Faster R-CNN tends to

give higher scores), so one fixed display threshold of 0.5 is not equally strict for both; and a box count above the number of annotated objects means some boxes are duplicates, false positives or objects the annotators did not label.

3.5 Comparison with Published Results

Table 5: Our mAP (x100) next to the published full-val2017 figures

Model	Published mAP (5,000 images)	Our mAP (100 images)	Source
FasterRCNN-R50-FPN-v2	46.7	56.0	[7] torchvision weights metadata
FasterRCNN-MobileNetV3-FPN	32.8	39.1	[7] torchvision weights metadata
YOLOv8n	37.3	45.3	[6] Ultralytics docs
YOLOv8s	44.9	53.8	[6] Ultralytics docs

Our values differ from the published ones because we evaluate a small subset; the published numbers use all 5,000 validation images. The ranking of the models is the useful part of the comparison. Published speeds are measured on GPUs with TensorRT or similar runtimes and are not comparable to our CPU timings.

3.6 Challenges Faced and Solutions

- **Different libraries, different outputs:** torchvision returns COCO category ids (1-90), YOLO returns class indices (0-79). We map indices to sorted COCO ids and assert the mapping against the annotation file.
- **Fair timing:** first inferences are slow and GPU calls are asynchronous. We added 10 un-timed warm-up images and synchronisation around every timed call, and excluded image loading from the timed region.
- **Fair accuracy:** detectors normally drop low-confidence boxes, which truncates the precision-recall curve. We set the score threshold to 0.001 and the maximum to 100 detections for both libraries.
- **Limited disk space and no GPU:** instead of downloading the 241 MB annotation archive, only the one JSON file is extracted with HTTP range requests, and on CPU the run uses 100 images and skips the largest YOLO model by default.
- **Reproducibility:** every number in this report is read from results.json, which the benchmark writes, so the report can be regenerated after any re-run.

4. Conclusion

4.1 Summary of What Was Achieved

We built a reproducible benchmark that compares two-stage and one-stage detectors under identical conditions and evaluated 4 pretrained models on 100 COCO val2017 images on CPU (x86_64, 4 threads). The headline result is that YOLOv8s delivered 33.2x the throughput of Faster R-CNN R50-FPN v2 with lower overall mAP (0.538 vs 0.560). The best overall accuracy came from FasterRCNN-R50-FPN-v2 and the best small-object accuracy from FasterRCNN-R50-FPN-v2. The classical rule "two-stage is accurate, one-stage is fast" still holds in direction: Faster R-CNN R50-FPN v2 leads YOLOv8s on mAP, mAP50, small-object mAP, medium-object mAP and they are level on mAP75, large-object mAP. However, the accuracy margin (2.2 mAP points) is small compared with the 33x speed difference, so for real-time use the one-stage detector is the better choice, and the two-stage detector is worth its cost only when accuracy, especially on small objects, matters more than speed. We also delivered a live side-by-side demo application.

4.2 Limitations

- Only 100 of the 5,000 COCO val2017 images were used, so mAP values have noticeable sampling variance.
- Only pretrained weights were evaluated; no model was trained or fine-tuned on a common recipe, so differences partly reflect training recipes (YOLOv8 is far newer than the original Faster R-CNN).
- Timings are hardware-dependent: they were measured on CPU (x86_64, 4 threads), where memory was shared with other applications, and will differ on other machines, especially on GPUs where the speed gap usually shrinks in absolute terms.
- No inference optimisation (TensorRT, ONNX Runtime, half precision, batching or quantisation) was used.
- Input resolutions differ (800 vs 640 pixels) because we used each model's default configuration.
- YOLOv8m was skipped in the CPU run to keep the run time short.

4.3 Possible Improvements

- Evaluate on all 5,000 validation images on a GPU and repeat timings several times to give confidence intervals.

- Export models to ONNX/TensorRT and use FP16 or INT8 to measure deployment-level speed.
- Compare at equal input resolution, and add other designs (RetinaNet, DETR / RT-DETR, YOLO newer versions).
- Fine-tune both families on the same custom dataset with the same budget for a training-controlled comparison.
- Measure energy and memory use, which matter for edge devices.

References

1. S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 91-99.
2. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779-788.
3. T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollar, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, Zurich, Switzerland, 2014, pp. 740-755.
4. T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 2117-2125.
5. R. Girshick, "Fast R-CNN," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1440-1448.
6. G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: https://docs.ultralytics.com/models/yolov8/ [Accessed: Nov. 8, 2026].
7. PyTorch Contributors, "Models and pre-trained weights - Object detection (torchvision)," PyTorch.org. [Online]. Available: https://docs.pytorch.org/vision/stable/models.html [Accessed: Nov. 8, 2026].
8. Y. Li, S. Xie, X. Chen, P. Dollar, K. He, and R. Girshick, "Benchmarking detection transfer learning with vision transformers," arXiv:2111.11429, 2021. (Training recipe behind torchvision's Faster R-CNN v2 weights.)
9. COCO Consortium, "COCO detection evaluation," cocodataset.org. [Online]. Available: https://cocodataset.org/#detection-eval [Accessed: Nov. 8, 2026].

Appendix

A. Source Code

GitHub repository: {{GITHUB_LINK}}

B. How to Run

The project is managed with uv (https://docs.astral.sh/uv/). uv reads pyproject.toml and uv.lock, creates the virtual environment and installs the exact package versions on first use.

```
uv sync          # one-time setup: environment + packages
uv run bench     # benchmark: downloads data + weights, writes results/
uv run build     # diagrams + this report + the slides
uv run demo      # live demo in the browser
```

C. Project Structure

```
cv26_rcnn_vs_yolo/
├── tasks.py           the short commands (uv run bench / report / slides / build /
demo)
├── common.py          model loading, unified detect(), timing, drawing
├── benchmark.py       COCO subset benchmark -> results/results.json + charts
├── figures.py         diagrams for report and slides
├── demo.py            Streamlit live demo (+ CLI fallback)
├── build_report.py    -> CV_Assignment2_Report.docx
├── build_slides.py    -> CV_Assignment2_Slides.pptx
├── pyproject.toml / uv.lock   dependencies (uv)
├── README.md
├── data/              COCO images + instances_val2017.json (downloaded)
├── weights/           model weights (downloaded)
└── results/           results.json, charts, sample images, diagrams
```

D. Experiment Environment

Device: CPU (x86_64, 4 threads); PyTorch threads: 4; run date: 2026-11-08T16:46:06; versions: python 3.13.5, torch 2.15.0+cpu, torchvision 0.29.2+cpu, ultralytics 8.4.207, pycocotools 2.0.11, opencv 5.0.0, numpy 2.5.4.